

8.7 操作进程的工具

Linux 系统提供了大量的监控和操作进程的有用工具。

STRACE：打印一个正在运行的程序和它的子进程调用的每个系统调用的轨迹。对于好奇的学生而言，这是一个令人着迷的工具。用-static 编译你的程序，能得到一个更干净的、不带有大量与共享库相关的输出的轨迹。

PS：列出当前系统中的进程(包括僵死进程)。

TOP：打印出关于当前进程资源使用的信息。

PMAP：显示进程的内存映射。

/proc：一个虚拟文件系统，以 ASCII 文本格式输出大量内核数据结构的内容，用户程序可以读取这些内容。比如，输入“cat/proc/loadavg”，可以看到你的 Linux 系统上当前的平均负载。

8.8 小结

异常控制流(ECF)发生在计算机系统的各个层次，是计算机系统中提供并发的基本机制。

在硬件层，异常是由处理器中的事件触发的控制流中的突变。控制流传递给一个软件处理程序，该处理程序进行一些处理，然后返回控制给被中断的控制流。

有四种不同类型的异常：中断、故障、终止和陷阱。当一个外部 I/O 设备(例如定时器芯片或者磁盘控制器)设置了处理器芯片上的中断管脚时，(对于任意指令)中断会异步地发生。控制返回到故障指令后面的那条指令。一条指令的执行可能导致故障和终止同步发生。故障处理程序会重新启动故障指令，而终止处理程序从不将控制返回给被中断的流。最后，陷阱就像是用来实现向应用提供到操作系统代码的受控的入口点的系统调用的函数调用。

在操作系统层，内核用 ECF 提供进程的基本概念。进程提供给应用两个重要的抽象：1)逻辑控制流，它提供给每个程序一个假象，好像它是在独占地使用处理器，2)私有地址空间，它提供给每个程序一个假象，好像它是在独占地使用主存。

在操作系统和应用程序之间的接口处，应用程序可以创建子进程，等待它们的子进程停止或者终止，运行新的程序，以及捕获来自其他进程的信号。信号处理的语义是微妙的，并且随系统不同而不同。然而，在与 Posix 兼容的系统上存在着一些机制，允许程序清楚地指定期望的信号处理语义。

最后，在应用层，C 程序可以使用非本地跳转来规避正常的调用/返回栈规则，并且直接从一个函数分支到另一个函数。

参考文献说明

Kerrisk 是 Linux 环境编程的完全参考手册[62]。Intel ISA 规范包含对 Intel 处理器上的异常和中断的详细讨论[50]。操作系统教科书[102, 106, 113]包括关于异常、进程和信号的其他信息。W. Richard Stevens 的[111]是一本有价值的和可读性很高的经典著作，是关于如何在应用程序中处理进程和信号的。Bovet 和 Cesati[11]给出了一个关于 Linux 内核的非常清晰的描述，包括进程和信号实现的细节。

家庭作业

• 8.9 考虑四个具有如下开始和结束时间的进程：

进程	开始时间	结束时间
A	5	7
B	2	4
C	3	6
D	1	8

对于每对进程，指明它们是否是并发地运行的：

进程对	并发地?
AB	
AC	
AD	
BC	
BD	
CD	

- * 8.10 在这一章里，我们介绍了一些具有不寻常的调用和返回行为的函数：setjmp、longjmp、execve 和 fork。找到下列行为中和每个函数相匹配的一种：

- A. 调用一次，返回两次。
- B. 调用一次，从不返回。
- C. 调用一次，返回一次或者多次。

- * 8.11 这个程序会输出多少个“hello”输出行？

```

1  #include "csapp.h"
2
3  int main()
4  {
5      int i;
6
7      for (i = 0; i < 2; i++)
8          Fork();
9      printf("hello\n");
10     exit(0);
11 }

```

code/ecf/forkprob1.c

- * 8.12 这个程序会输出多少个“hello”输出行？

```

1  #include "csapp.h"
2
3  void doit()
4  {
5      Fork();
6      Fork();
7      printf("hello\n");
8      return;
9  }
10
11 int main()
12 {
13     doit();
14     printf("hello\n");
15     exit(0);
16 }

```

code/ecf/forkprob4.c

- * 8.13 下面程序的一种可能的输出是什么？

```

1  #include "csapp.h"
2
3  int main()

```

code/ecf/forkprob3.c

```

4  {
5      int x = 3;
6
7      if (Fork() != 0)
8          printf("x=%d\n", ++x);
9
10     printf("x=%d\n", --x);
11     exit(0);
12 }

```

code/ecf/forkprob3.c

* 8.14 下面这个程序会输出多少个“hello”输出行？

code/ecf/forkprob5.c

```

1  #include "csapp.h"
2
3  void doit()
4  {
5      if (Fork() == 0) {
6          Fork();
7          printf("hello\n");
8          exit(0);
9      }
10     return;
11 }
12
13 int main()
14 {
15     doit();
16     printf("hello\n");
17     exit(0);
18 }

```

code/ecf/forkprob5.c

* 8.15 下面这个程序会输出多少个“hello”输出行？

code/ecf/forkprob6.c

```

1  #include "csapp.h"
2
3  void doit()
4  {
5      if (Fork() == 0) {
6          Fork();
7          printf("hello\n");
8          return;
9      }
10     return;
11 }
12
13 int main()
14 {
15     doit();
16     printf("hello\n");
17     exit(0);
18 }

```

code/ecf/forkprob6.c

* 8.16 下面这个程序的输出是什么？

code/ecf/forkprob7.c

```

1  #include "csapp.h"
2  int counter = 1;
3
4  int main()
5  {
6      if (fork() == 0) {
7          counter--;
8          exit(0);
9      }
10     else {
11         Wait(NULL);
12         printf("counter = %d\n", ++counter);
13     }
14     exit(0);
15 }

```

code/ecf/forkprob7.c

* 8.17 列举练习题 8.4 中程序所有可能的输出。

** 8.18 考虑下面的程序：

code/ecf/forkprob2.c

```

1  #include "csapp.h"
2
3  void end(void)
4  {
5      printf("2"); fflush(stdout);
6  }
7
8  int main()
9  {
10     if (Fork() == 0)
11         atexit(end);
12     if (Fork() == 0) {
13         printf("0"); fflush(stdout);
14     }
15     else {
16         printf("1"); fflush(stdout);
17     }
18     exit(0);
19 }

```

code/ecf/forkprob2.c

判断下面哪个输出是可能的。注意：atexit 函数以一个指向函数的指针为输入，并将它添加到函数列表中(初始为空)，当 exit 函数被调用时，会调用该列表中的函数。

A. 112002 B. 211020 C. 102120 D. 122001 E. 100212

** 8.19 下面的函数会打印多少行输出？用一个 n 的函数给出答案。假设 $n \geq 1$ 。

code/ecf/forkprob8.c

```

1  void foo(int n)
2  {
3      int i;
4
5      for (i = 0; i < n; i++)
6          Fork();
7      printf("hello\n");
8      exit(0);
9  }

```

code/ecf/forkprob8.c

- ** 8.20 使用 `execve` 编写一个叫做 `myls` 的程序，该程序的行为和 `/bin/ls` 程序的一样。你的程序应该接受相同的命令行参数，解释同样的环境变量，并产生相同的输出。

`ls` 程序从 `COLUMNS` 环境变量中获得屏幕的宽度。如果没有设置 `COLUMNS`，那么 `ls` 会假设屏幕宽 80 列。因此，你可以通过把 `COLUMNS` 环境设置得小于 80，来检查你对环境变量的处理：

```
linux> setenv COLUMNS 40
linux> ./mysls
: // Output is 40 columns wide
linux> unsetenv COLUMNS
linux> ./mysls
: // Output is now 80 columns wide
```

- ** 8.21 下面的程序可能的输出序列是什么？

```
----- code/ecf/waitprob3.c
1  int main()
2  {
3      if (fork() == 0) {
4          printf("a"); fflush(stdout);
5          exit(0);
6      }
7      else {
8          printf("b"); fflush(stdout);
9          waitpid(-1, NULL, 0);
10     }
11     printf("c"); fflush(stdout);
12     exit(0);
13 }
```

----- code/ecf/waitprob3.c

- ** 8.22 编写 Unix `system` 函数的你自己的版本

```
int mysystem(char *command);
```

`mysystem` 函数通过调用 `"/bin/sh -c command"` 来执行 `command`，然后在 `command` 完成后返回。如果 `command` (通过调用 `exit` 函数或者执行一条 `return` 语句) 正常退出，那么 `mysystem` 返回 `command` 退出状态。例如，如果 `command` 通过调用 `exit(8)` 终止，那么 `mysystem` 返回值 8。否则，如果 `command` 是异常终止的，那么 `mysystem` 就返回 `shell` 返回的状态。

- ** 8.23 你的一个同事想要使用信号来让一个父进程对发生在子进程中的事件计数。其想法是每次发生一个事件时，通过向父进程发送一个信号来通知它，并且让父进程的信号处理程序对一个全局变量 `counter` 加一，在子进程终止之后，父进程就可以检查这个变量。然而，当他在系统上运行图 8-45 中的测试程序时，发现当父进程调用 `printf` 时，`counter` 的值总是 2，即使子进程向父进程发送了 5 个信号也是如此。他很困惑，向你寻求帮助。你能解释这个程序有什么错误吗？

```
----- code/ecf/counterprob.c
1  #include "csapp.h"
2
3  int counter = 0;
4
5  void handler(int sig)
6  {
7      counter++;
8      sleep(1); /* Do some work in the handler */
9      return;
```

图 8-45 家庭作业 8.23 中引用的计数器程序

```

10 }
11
12 int main()
13 {
14     int i;
15
16     Signal(SIGUSR2, handler);
17
18     if (Fork() == 0) { /* Child */
19         for (i = 0; i < 5; i++) {
20             Kill(getppid(), SIGUSR2);
21             printf("sent SIGUSR2 to parent\n");
22         }
23         exit(0);
24     }
25
26     Wait(NULL);
27     printf("counter=%d\n", counter);
28     exit(0);
29 }

```

code/ecf/counterprob.c

图 8-45 (续)

****8.24** 修改图 8-18 中的程序，以满足下面两个条件：

- 1) 每个子进程在试图写一个只读文本段中的位置时会异常终止。
- 2) 父进程打印和下面所示相同(除了 PID)的输出：

```

child 12255 terminated by signal 11: Segmentation fault
child 12254 terminated by signal 11: Segmentation fault

```

提示：请参考 `psignal(3)` 的 man 页。

****8.25** 编写 `fgets` 函数的一个版本，叫做 `tfgets`，它 5 秒钟后会超时。`tfgets` 函数接收和 `fgets` 相同的输入。如果用户在 5 秒内不键入一个输入行，`tfgets` 返回 `NULL`。否则，它返回一个指向输入行的指针。

****8.26** 以图 8-23 中的示例作为开始点，编写一个支持作业控制的 shell 程序。shell 必须具有以下特性：

- 用户输入的命令行由一个 name、零个或者多个参数组成，它们都由一个或者多个空格分隔开。如果 name 是一个内置命令，那么 shell 就立即处理它，并等待下一个命令行。否则，shell 就假设 name 是一个可执行文件，在一个初始的子进程(作业)的上下文中加载并运行它。作业的进程组 ID 与子进程的 PID 相同。
- 每个作业是由一个进程 ID(PID)或者一个作业 ID(JID)来标识的，它是由一个 shell 分配的任意的小正整数。JID 在命令行上用前缀“%”来表示。比如，“%5”表示 JID 5，而“5”表示 PID 5。
- 如果命令行以 & 来结束，那么 shell 就在后台运行这个作业。否则，shell 就在前台运行这个作业。
- 输入 `Ctrl+C`(`Ctrl+Z`)，使得内核发送一个 `SIGINT`(`SIGTSTP`)信号给 shell，shell 再转发给前台进程组中的每个进程[⊖]。
- 内置命令 `jobs` 列出所有的后台作业。
- 内置命令 `bg job` 通过发送一个 `SIGCONT` 信号重启 `job`，然后在后台运行它。`job` 参数可以是一个 PID，也可以是一个 JID。
- 内置命令 `fg job` 通过发送一个 `SIGCONT` 信号重启 `job`，然后在前台运行它。

⊖ 注意这是对真实的 shell 工作方式的简化。真实的 shell 里，内核响应 `Ctrl+C`(`Ctrl+Z`)，把 `SIGINT`(`SIGTSTP`)直接发送给终端前台进程组中的每个进程。shell 用 `tcsetpgrp` 函数管理这个进程组的成员，用 `tcsetattr` 函数管理终端的属性，这两个函数都超出了本书讲述的范围。可以参考[62]获得详细信息。

